

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA5112

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

Coding Challenge

Code of Conduct:

I G. Srivalli (192365014) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

G. Srivalli

Annexure A

Coding Challenge

Coding Challenge (Additional Set)

- Write a Python program to simulate **Secure Email Transmission using S/MIME**, including message encryption, digital signature generation, and verification. Analyze how confidentiality and authentication are ensured.
- Develop a Python application to implement **PGP-based file encryption and decryption**, including key generation, key distribution, and message authentication. Evaluate how PGP provides both security and efficiency.
- Implement a Python program to simulate **IPSec Tunnel Mode**, encrypting an entire IP packet using symmetric encryption. Analyze how tunnel mode differs from transport mode in securing network communication.
- Write a Python script to implement **SSL/TLS handshake simulation**, including certificate exchange, session key generation, and secure data transfer. Evaluate how the handshake ensures secure web communication.
- Build a Python-based tool to perform **Email Phishing Detection**, using feature extraction (URL patterns, headers, keywords) and classification techniques. Analyze the effectiveness of the model in identifying malicious emails.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Question 1: Secure Email Transmission Using S/MIME Simulation

1. Concept & Theoretical Background:

Secure/Multipurpose Internet Mail Extensions (S/MIME) provides end-to-end cryptographic security for electronic messaging. It implements a hybrid cryptosystem combining public-key cryptography (asymmetric) for digital signatures and session key exchange, and symmetric-key cryptography for bulk payload encryption. S/MIME ensures the four cornerstones of information security: Confidentiality, Authentication, Integrity, and Non-Repudiation.

2. Python Implementation:

```
import os
import json
import base64
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers import Cipher,
algorithms, modes
from cryptography.hazmat.backends import default_backend

class SMIMESimulator:
    def __init__(self):
        # Generate Sender (Alice) Keypair for Digital Signatures
        self.alice_private =
rsa.generate_private_key(public_exponent=65537, key_size=2048)
        self.alice_public = self.alice_private.public_key()

        # Generate Receiver (Bob) Keypair for Key Encapsulation
        self.bob_private =
rsa.generate_private_key(public_exponent=65537, key_size=2048)
        self.bob_public = self.bob_private.public_key()

    def send_secure_email(self, message: str) -> dict:
        """
        Simulates S/MIME sending:
        1. Sign the plaintext using Sender's Private Key.
        2. Generate ephemeral AES-256 session key and encrypt
message + signature.
        3. Encrypt AES session key with Receiver's Public Key.
        """
```

```

        msg_bytes = message.encode('utf-8')

        # Step 1: Digital Signature Generation (Authentication & Integrity)
        signature = self.alice_private.sign(
            msg_bytes,
            padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH),
            hashes.SHA256()
        )

        payload = json.dumps({
            "plaintext": message,
            "signature": base64.b64encode(signature).decode('utf-8')
        }).encode('utf-8')

        # Step 2: Symmetric Bulk Encryption (Confidentiality)
        aes_session_key = os.urandom(32) # AES-256
        iv = os.urandom(16) # AES CBC IV

        # PKCS#7 Padding
        pad_len = 16 - (len(payload) % 16)
        padded_payload = payload + bytes([pad_len] * pad_len)

        cipher = Cipher(algorithms.AES(aes_session_key),
modes.CBC(iv), backend=default_backend())
        encryptor = cipher.encryptor()
        encrypted_payload = encryptor.update(padded_payload) +
encryptor.finalize()

        # Step 3: Asymmetric Session Key Encryption
        encrypted_session_key = self.bob_public.encrypt(
            aes_session_key,
            padding.OAEP(
                mgf=padding.MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None
            )
        )

    return {
        "encrypted_payload":

```

```

base64.b64encode(encrypted_payload).decode('utf-8'),
        "iv": base64.b64encode(iv).decode('utf-8'),
        "encrypted_session_key":
base64.b64encode(encrypted_session_key).decode('utf-8')
    }

    def receive_secure_email(self, package: dict) -> dict:
        """
        Simulates S/MIME receiving:
        1. Decrypt AES session key with Receiver's Private Key.
        2. Decrypt ciphertext payload using decrypted AES key.
        3. Verify Digital Signature with Sender's Public Key.
        """
        encrypted_session_key =
base64.b64decode(package["encrypted_session_key"])
        iv = base64.b64decode(package["iv"])
        encrypted_payload =
base64.b64decode(package["encrypted_payload"])

        # Step 1: Decrypt Session Key
        session_key = self.bob_private.decrypt(
            encrypted_session_key,
            padding.OAEP(
                mgf=padding.MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None
            )
        )

        # Step 2: Decrypt Payload
        cipher = Cipher(algorithms.AES(session_key), modes.CBC(iv),
backend=default_backend())
        decryptor = cipher.decryptor()
        padded_payload = decryptor.update(encrypted_payload) +
decryptor.finalize()

        pad_len = padded_payload[-1]
        payload = json.loads(padded_payload[:-pad_len].decode('utf-
8'))

        received_msg = payload["plaintext"].encode('utf-8')
        signature = base64.b64decode(payload["signature"])

```

```

        # Step 3: Verify Digital Signature
        try:
            self.alice_public.verify(
                signature,
                received_msg,
                padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH),
                hashes.SHA256()
            )
            verification_status = "VERIFIED (Authentic &
Untampered)"
        except Exception as e:
            verification_status = f"FAILED: {str(e)}"

        return {
            "message": payload["plaintext"],
            "verification": verification_status
        }

# --- Execution Example ---
if __name__ == "__main__":
    smime = SMIMESimulator()
    original_text = "CONFIDENTIAL: Semester Exam Question Paper
CSA51 - Cryptography."
    print("Original Text:", original_text)

    # Send
    email_pkg = smime.send_secure_email(original_text)
    print("\nEncrypted Transmission Package:")
    print("Encrypted Session Key:",
email_pkg["encrypted_session_key"][:30], "...")
    print("Encrypted Body:", email_pkg["encrypted_payload"][:30],
"...")

    # Receive
    result = smime.receive_secure_email(email_pkg)
    print("\nReceived & Decrypted:")
    print("Decrypted Message:", result["message"])
    print("Signature Status:", result["verification"])

```

3. Security Analysis: Confidentiality and Authentication

- **How Confidentiality is Ensured:** Confidentiality is achieved through symmetric encryption (AES-256 in CBC mode). Bulk symmetric encryption protects the contents against eavesdropping over insecure mail transport networks. The one-time session key is protected via asymmetric RSA-OAEP encryption using the receiver's public key. Only the legitimate recipient with the corresponding private key can decrypt the session key and subsequently the email content.

- **How Authentication & Integrity are Ensured:** Authentication is established using digital signatures (RSA-PSS with SHA-256). The sender hashes the plaintext and encrypts the digest with their private key. The recipient decrypts the signature using the sender's public key and verifies that the recalculated hash matches the transmitted digest. This confirms message integrity (no modification) and authenticates the sender's identity, preventing spoofing and repudiation.

Question 2: PGP-Based File Encryption, Key Distribution & Authentication

1. Concept & Architecture:

Pretty Good Privacy (PGP) combines symmetric key encryption, public-key cryptography, data compression (ZIP), and hashing/digital signatures. PGP operates on the 'Web of Trust' model for decentralized key distribution, avoiding single-point CA bottlenecks while ensuring robust mutual validation.

2. Python Implementation:

```
import os
import zlib
import json
import base64
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

class PGPFileSecuritySystem:
    def __init__(self, user_name: str):
        self.user_name = user_name
        self.private_key =
rsa.generate_private_key(public_exponent=65537, key_size=2048)
        self.public_key = self.private_key.public_key()
        self.keyring = {} # Web of Trust Keyring

    def export_public_key(self) -> str:
        """Export ASCII-armored Public Key for Key Distribution."""
        pem = self.public_key.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        )
        return pem.decode('utf-8')

    def import_peer_key(self, peer_name: str, pem_str: str):
        """Import and store peer's public key in local keyring."""
        peer_pub =
serialization.load_pem_public_key(pem_str.encode('utf-8'))
        self.keyring[peer_name] = peer_pub
```

```

def pgp_encrypt_file(self, recipient_name: str, file_bytes:
bytes) -> dict:
    """
    PGP Process:
    1. Sign data (SHA-256 + RSA).
    2. Compress signed payload with Deflate/Zlib.
    3. Encrypt compressed data with AES-256-GCM.
    4. Encrypt session key with Recipient's Public Key.
    """
    if recipient_name not in self.keyring:
        raise ValueError(f"Recipient {recipient_name}'s public
key not found in keyring.")

    recipient_pub = self.keyring[recipient_name]

    # Step 1: Digital Signature
    signature = self.private_key.sign(
        file_bytes,
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH),
        hashes.SHA256()
    )

    package = json.dumps({
        "data": base64.b64encode(file_bytes).decode('utf-8'),
        "signature": base64.b64encode(signature).decode('utf-
8'),
        "sender": self.user_name
    }).encode('utf-8')

    # Step 2: Compression (Efficiency)
    compressed_package = zlib.compress(package, level=9)

    # Step 3: Symmetric Encryption (AES-GCM for Authenticated
Encryption)
    session_key = AESGCM.generate_key(bit_length=256)
    aesgcm = AESGCM(session_key)
    nonce = os.urandom(12)
    encrypted_data = aesgcm.encrypt(nonce, compressed_package,
None)

    # Step 4: Asymmetric Session Key Encryption

```

```

        encrypted_session_key = recipient_pub.encrypt(
            session_key,
            padding.OAEP(
                mgf=padding.MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None
            )
        )

    return {
        "enc_session_key":
base64.b64encode(encrypted_session_key).decode('utf-8'),
        "nonce": base64.b64encode(nonce).decode('utf-8'),
        "ciphertext":
base64.b64encode(encrypted_data).decode('utf-8')
    }

def pgp_decrypt_file(self, pgp_packet: dict) -> dict:
    """
    PGP Decryption:
    1. Decrypt session key using Private Key.
    2. Decrypt ciphertext payload using AES-GCM.
    3. Decompress data.
    4. Authenticate signature using Sender's Public Key from
Keyring.
    """
    enc_session_key =
base64.b64decode(pgp_packet["enc_session_key"])
    nonce = base64.b64decode(pgp_packet["nonce"])
    ciphertext = base64.b64decode(pgp_packet["ciphertext"])

    # Step 1: Recover Session Key
    session_key = self.private_key.decrypt(
        enc_session_key,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )

    # Step 2: Decrypt Payload

```

```

aesgcm = AESGCM(session_key)
compressed_package = aesgcm.decrypt(nonce, ciphertext, None)

# Step 3: Decompress
package_bytes = zlib.decompress(compressed_package)
package = json.loads(package_bytes.decode('utf-8'))

sender = package["sender"]
data = base64.b64decode(package["data"])
sig = base64.b64decode(package["signature"])

# Step 4: Verify Signature
if sender not in self.keyring:
    sig_status = f"FAILED: Sender '{sender}' not in
Keyring."
else:
    sender_pub = self.keyring[sender]
    try:
        sender_pub.verify(
            sig,
            data,
            padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH),
            hashes.SHA256()
        )
        sig_status = f"VERIFIED: Authentic signature from
{sender}"
    except Exception as e:
        sig_status = f"FAILED: {str(e)}"

    return {"data": data, "sender": sender, "status":
sig_status}

# --- Execution Example ---
if __name__ == "__main__":
    alice = PGPSecuritySystem("Alice")
    bob = PGPSecuritySystem("Bob")

    # Key Distribution (Public Key Exchange)
    alice.import_peer_key("Bob", bob.export_public_key())
    bob.import_peer_key("Alice", alice.export_public_key())

```

```
file_content = b"CRITICAL_RESEARCH_DATA: Proprietary  
cryptographic algorithm details."  
encrypted_file = alice.pgp_encrypt_file("Bob", file_content)  
  
print("File Encrypted Successfully. Packet generated.")  
decrypted_result = bob.pgp_decrypt_file(encrypted_file)  
print("Decrypted Content:",  
decrypted_result["data"].decode('utf-8'))  
print("Signature Status:", decrypted_result["status"])
```

3. Evaluation: Security & Efficiency Trade-offs:

- 1. Security:** PGP uses hybrid cryptography ensuring confidentiality via high-speed symmetric ciphers (AES-256) and key exchange via RSA-2048. Digital signatures provide non-repudiation and tampering detection.
- 2. Efficiency:** Asymmetric operations are computationally heavy. PGP only applies RSA on small data (the 256-bit symmetric session key and the SHA-256 digest). Bulk data is processed via hardware-accelerated AES.
- 3. Cryptographic Advantage of Compression:** Compressing data before encryption reduces redundant patterns and language entropy, significantly mitigating frequency-analysis cryptanalytic attacks while reducing network transmission overhead.

Question 3: IPSec Tunnel Mode Simulation & Mode Comparison

1. Concept & Architectural Difference:

IPSec (Internet Protocol Security) operates at the Network Layer (Layer 3). In Transport Mode, only the IP payload (data/transport layer) is encrypted, keeping the original IP header visible. In Tunnel Mode, the entire original IP packet (both original IP header and payload) is encrypted and encapsulated within a new outer IP packet and ESP (Encapsulating Security Payload) header/trailer. This hides internal source/destination subnet topologies.

2. Python Implementation:

```
import os
import json
import base64
import hashlib
import hmac
from cryptography.hazmat.primitives.ciphers import Cipher,
algorithms, modes
from cryptography.hazmat.backends import default_backend
class IPPacket:
    def __init__(self, src_ip: str, dst_ip: str, protocol: str,
payload: str):
        self.header = {"src_ip": src_ip, "dst_ip": dst_ip,
"protocol": protocol, "ttl": 64}
        self.payload = payload
    def serialize(self) -> bytes:
        return json.dumps({"header": self.header, "payload":
self.payload}).encode('utf-8')
    @classmethod
    def deserialize(cls, raw_bytes: bytes):
        data = json.loads(raw_bytes.decode('utf-8'))
        pkt = cls(data["header"]["src_ip"],
data["header"]["dst_ip"],
data["header"]["protocol"], data["payload"])
        pkt.header["ttl"] = data["header"]["ttl"]
        return pkt
class IPsecTunnelGateway:
    def __init__(self, gateway_ip: str, peer_gateway_ip: str,
shared_aes_key: bytes, shared_hmac_key: bytes):
        self.gw_ip = gateway_ip
```

```

        self.peer_gw_ip = peer_gateway_ip
        self.aes_key = shared_aes_key
        self.hmac_key = shared_hmac_key
        self.spi = 0x1000A1 # Security Parameters Index
        self.seq_num = 1
    def encapsulate_and_encrypt(self, original_packet: IPPacket) -> dict:
        """
        IPSec Tunnel Mode Encapsulation:
        1. Serialize entire original packet (Header + Payload).
        2. Apply AES-CBC encryption with PKCS#7 padding.
        3. Prepend ESP Header & Append ESP Trailer + HMAC Auth Data.
        4. Encapsulate inside New Outer IP Header (Gateway-to-
Gateway).
        """
        raw_inner_packet = original_packet.serialize()
        # PKCS#7 Padding
        pad_len = 16 - (len(raw_inner_packet) % 16)
        padded_inner = raw_inner_packet + bytes([pad_len] * pad_len)
        # Encrypt inner packet
        iv = os.urandom(16)
        cipher = Cipher(algorithms.AES(self.aes_key), modes.CBC(iv),
backend=default_backend())
        encryptor = cipher.encryptor()
        encrypted_payload = encryptor.update(padded_inner) +
encryptor.finalize()

        esp_header = {"spi": hex(self.spi), "seq": self.seq_num,
"iv": base64.b64encode(iv).decode('utf-8')}
        self.seq_num += 1

        # Integrity / Authentication Data (HMAC-SHA256 over ESP
header + ciphertext)
        auth_data = hmac.new(self.hmac_key, encrypted_payload +
str(esp_header).encode(), hashlib.sha256).hexdigest()

        # New Outer IP Header
        outer_ip_header = {
            "src_gw": self.gw_ip,
            "dst_gw": self.peer_gw_ip,
            "protocol": "ESP (Protocol 50)"
        }

```

```

        return {
            "outer_header": outer_ip_header,
            "esp_header": esp_header,
            "esp_payload_ciphertext":
base64.b64encode(encrypted_payload).decode('utf-8'),
            "esp_auth_icv": auth_data
        }

    def decrypt_and_decapsulate(self, tunneled_packet: dict) ->
IPPacket:
        """
        Decapsulation at destination gateway:
        1. Verify ESP ICV Authentication.
        2. Decrypt encrypted payload using AES-CBC.
        3. Reconstruct original IP Packet.
        """

        encrypted_payload =
base64.b64decode(tunneled_packet["esp_payload_ciphertext"])
        esp_header = tunneled_packet["esp_header"]
        iv = base64.b64decode(esp_header["iv"])
        icv = tunneled_packet["esp_auth_icv"]

        # Verify HMAC
        expected_icv = hmac.new(self.hmac_key, encrypted_payload +
str(esp_header).encode(), hashlib.sha256).hexdigest()
        if not hmac.compare_digest(icv, expected_icv):
            raise ValueError("Integrity verification failed! ICV
mismatch.")

        # Decrypt
        cipher = Cipher(algorithms.AES(self.aes_key), modes.CBC(iv),
backend=default_backend())
        decryptor = cipher.decryptor()
        padded_inner = decryptor.update(encrypted_payload) +
decryptor.finalize()

        pad_len = padded_inner[-1]
        raw_inner = padded_inner[:-pad_len]

        return IPPacket.deserialize(raw_inner)

```



```
# --- Execution Example ---
if __name__ == "__main__":
    aes_k = os.urandom(32)
    hmac_k = os.urandom(32)

    gw_site_a = IPsecTunnelGateway("203.0.113.1", "198.51.100.1",
aes_k, hmac_k)
    gw_site_b = IPsecTunnelGateway("198.51.100.1", "203.0.113.1",
aes_k, hmac_k)

    # Internal Host A -> Internal Host B
    orig_pkt = IPPacket(src_ip="192.168.1.50", dst_ip="10.0.0.99",
protocol="TCP", payload="SALARY_RECORDS_CONFIDENTIAL")

    tunneled = gw_site_a.encapsulate_and_encrypt(orig_pkt)
    print("Tunneled Packet on Public Internet:",
json.dumps(tunneled, indent=2))

    recovered_pkt = gw_site_b.decrypt_and_decapsulate(tunneled)
    print("\nDecapsulated Original Packet:")
    print("Source IP:", recovered_pkt.header["src_ip"])
    print("Destination IP:", recovered_pkt.header["dst_ip"])
    print("Payload:", recovered_pkt.payload)
```

3. Analysis: Tunnel Mode vs. Transport Mode

Feature	IPSec Transport Mode	IPSec Tunnel Mode
Protected Scope	Only IP Payload (Transport Layer + Data)	Entire IP Packet (Original Header + Payload)
IP Header Exposure	Original IP Header is fully visible to observers	Original IP Header is encrypted & hidden; new outer header added
Use Case Scenario	Host-to-Host (End-to-End between two endpoints)	Gateway-to-Gateway (Site-to-Site VPNs) / Host-to-Gateway
Traffic Flow Security	Vulnerable to traffic analysis (reveals end-node IPs)	Protects against traffic analysis of internal corporate topology

Question 4: SSL/TLS Handshake Simulation & Secure Web Communication

1. Concept & TLS Handshake Steps:

The SSL/TLS handshake protocol establishes a secure, encrypted communication channel over an untrusted transport layer (TCP/IP). Key phases include: (1) ClientHello & ServerHello (cipher suite negotiation & random nonces), (2) Server Certificate & Public Key delivery, (3) Pre-Master Secret exchange encrypted via RSA / Diffie-Hellman, (4) Symmetric Session Key Derivation (PRF/HKDF), and (5) Secure Symmetric Application Data Exchange.

2. Python Implementation:

```
import os
import base64
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

class TLSServer:
    def __init__(self, domain: str):
        self.domain = domain
        self.private_key =
rsa.generate_private_key(public_exponent=65537, key_size=2048)
        self.public_key = self.private_key.public_key()
        self.server_random = None
        self.session_key = None

    def receive_client_hello(self, client_random: bytes,
cipher_suites: list) -> dict:
        self.client_random = client_random
        self.server_random = os.urandom(32)
        self.selected_cipher = "TLS_AES_256_GCM_SHA384"

        # Simulated Certificate payload containing Server Public Key
        certificate = {
            "subject": f"CN={self.domain}",
            "issuer": "CN=Trusted SIMATS Root CA",
            "public_key": self.public_key
        }
        return {
```

```

        "server_random": self.server_random,
        "selected_cipher": self.selected_cipher,
        "certificate": certificate
    }

    def complete_handshake(self, enc_pre_master_secret: bytes):
        # Decrypt Pre-Master Secret using Server Private Key
        pre_master_secret = self.private_key.decrypt(
            enc_pre_master_secret,

padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()),
algorithm=hashes.SHA256(), label=None)
        )

        # Derive Session Key using HKDF (PRF)
        hkdf = HKDF(
            algorithm=hashes.SHA256(),
            length=32,
            salt=self.client_random + self.server_random,
            info=b"tls_session_master_secret"
        )
        self.session_key = hkdf.derive(pre_master_secret)

    def decrypt_application_data(self, nonce: bytes, ciphertext:
bytes) -> str:
        aesgcm = AESGCM(self.session_key)
        return aesgcm.decrypt(nonce, ciphertext, None).decode('utf-
8')

class TLSClient:
    def __init__(self):
        self.client_random = os.urandom(32)
        self.session_key = None

    def start_handshake(self) -> tuple:
        ciphers = ["TLS_AES_256_GCM_SHA384",
"TLS_CHACHA20_POLY1305_SHA256"]
        return self.client_random, ciphers

    def process_server_hello(self, server_hello_pkg: dict) -> bytes:
        self.server_random = server_hello_pkg["server_random"]
        server_cert = server_hello_pkg["certificate"]

```

```

server_pub_key = server_cert["public_key"]

# Generate 48-byte Pre-Master Secret
pre_master_secret = os.urandom(48)

# Encrypt Pre-Master Secret with Server's Public Key
enc_pre_master = server_pub_key.encrypt(
    pre_master_secret,

padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()),
algorithm=hashes.SHA256(), label=None)
)

# Derive Session Key using same HKDF PRF
hkdf = HKDF(
    algorithm=hashes.SHA256(),
    length=32,
    salt=self.client_random + self.server_random,
    info=b"tls_session_master_secret"
)
self.session_key = hkdf.derive(pre_master_secret)
return enc_pre_master

def encrypt_application_data(self, plaintext: str) -> tuple:
    aesgcm = AESGCM(self.session_key)
    nonce = os.urandom(12)
    ciphertext = aesgcm.encrypt(nonce, plaintext.encode('utf-
8'), None)
    return nonce, ciphertext

# --- Execution Example ---
if __name__ == "__main__":
    client = TLSClient()
    server = TLSServer("secure.simats.edu")

    # 1. ClientHello
    c_rand, c_ciphers = client.start_handshake()

    # 2. ServerHello & Certificate
    s_response = server.receive_client_hello(c_rand, c_ciphers)

    # 3. Client Key Exchange

```

```
enc_pms = client.process_server_hello(s_response)

# 4. Server Finished
server.complete_handshake(enc_pms)
print("TLS Handshake Completed Successfully! Derived Session
Keys Match.")

# 5. Encrypted Application Data Transfer
http_req = "GET /student/results HTTP/1.1\r\nHost:
secure.simats.edu\r\nAuthorization: Bearer Token123"
nonce, ct = client.encrypt_application_data(http_req)

# Server Receives & Decrypts
decrypted_http = server.decrypt_application_data(nonce, ct)
print("\nDecrypted HTTP Request on Server:\n", decrypted_http)
```

3. Evaluation: How the Handshake Secures Web Communication

- 1. Mutual Authentication & Anti-Spoofing:** Digital certificates bind the domain name to the server's public key, verified by trusted Certificate Authorities (CAs). This prevents Man-in-the-Middle (MitM) and DNS spoofing attacks.
- 2. Dynamic Session Key Derivation:** Both sides contribute cryptographically secure random nonces (ClientRandom, ServerRandom) combined with the Pre-Master Secret. This ensures freshness and thwarts replay attacks.
- 3. Authenticated Symmetric Data Channel:** Once established, TLS uses AEAD (AES-GCM / ChaCha20-Poly1305) to provide simultaneous line-speed encryption and authentication tags for every exchanged frame.

Question 5: Python-Based Email Phishing Detection System

1. Architecture & Feature Engineering Methodology:

Phishing email detection is formulated as a supervised machine learning classification task. A hybrid feature vector is constructed by combining: (1) NLP TF-IDF text features, (2) Domain / URL anomaly heuristics (IP addresses in URLs, excessive subdomains, shortening services), and (3) Header metadata (DKIM/SPF spoofing indicators, mismatch between From and Reply-To headers).

2. Python Implementation:

```
import re
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score
from scipy.sparse import hstack, csr_matrix

class EmailPhishingDetector:
    def __init__(self):
        self.vectorizer = TfidfVectorizer(max_features=250,
stop_words='english')
        self.classifier = RandomForestClassifier(n_estimators=100,
random_state=42)

    def extract_handcrafted_features(self, emails: list) ->
np.ndarray:
        """
        Extracts structural, header, and URL heuristic features:
        1. Urgent keyword count
        2. URL count
        3. Presence of IP addresses in links
        4. Suspicious TLD / domain spoofing indicators
        5. Suspicious header mismatches (From vs Reply-To)
        """
        features = []
        urgent_pattern =
re.compile(r'(urgent|verify|suspend|immediately|password|account
locked|update banking)', re.I)
        ip_url_pattern = re.compile(r'http[s]?://(?:[0-
9]{1,3}\.){3}[0-9]{1,3}')
```

```

        for email in emails:
            text = email.get("body", "")
            headers = email.get("headers", {})
            urls = re.findall(r'http[s]?://(?:[a-zA-Z]|[0-9]|[$-
            _@.&+]|[*%\(\)\,]|(?:%[0-9a-fA-F][0-9a-fA-F]))+', text)

            urgent_cnt = len(urgent_pattern.findall(text))
            url_cnt = len(urls)
            has_ip_url = 1 if ip_url_pattern.search(text) else 0

            # Header anomaly: Reply-To mismatch
            sender = headers.get("From", "")
            reply_to = headers.get("Reply-To", sender)
            header_mismatch = 1 if sender != reply_to else 0

            features.append([urgent_cnt, url_cnt, has_ip_url,
            header_mismatch])

        return np.array(features)

    def train(self, email_dataset: list, labels: list):
        bodies = [e.get("body", "") for e in email_dataset]
        tfidf_matrix = self.vectorizer.fit_transform(bodies)
        handcrafted_matrix =
        csr_matrix(self.extract_handcrafted_features(email_dataset))

        # Combine NLP TF-IDF and heuristic feature matrices
        X = hstack([tfidf_matrix, handcrafted_matrix])
        self.classifier.fit(X, labels)

    def predict(self, email_dataset: list) -> np.ndarray:
        bodies = [e.get("body", "") for e in email_dataset]
        tfidf_matrix = self.vectorizer.transform(bodies)
        handcrafted_matrix =
        csr_matrix(self.extract_handcrafted_features(email_dataset))

        X = hstack([tfidf_matrix, handcrafted_matrix])
        return self.classifier.predict(X)

# --- Execution Example ---
if __name__ == "__main__":
    train_emails = [

```

```

        {"headers": {"From": "security@paypal.com", "Reply-To":
"hacker@192.168.1.1"},
        "body": "URGENT: Your account is suspended. Verify password
at http://192.168.1.100/login immediately."},
        {"headers": {"From": "prof@simats.edu", "Reply-To":
"prof@simats.edu"},
        "body": "Reminder: The Cryptography and Network Security
assessment is due tomorrow. Check portal."},
        {"headers": {"From": "alert@banking.com", "Reply-To":
"phish@fake.com"},
        "body": "Update banking info immediately or your card will
be blocked. Click here http://10.0.0.1/verify"},
        {"headers": {"From": "newsletter@tech.org", "Reply-To":
"newsletter@tech.org"},
        "body": "Here is your weekly summary of artificial
intelligence and cybersecurity trends."}
    ]
    train_labels = [1, 0, 1, 0] # 1: Phishing, 0: Legitimate

    detector = EmailPhishingDetector()
    detector.train(train_emails, train_labels)

    test_emails = [
        {"headers": {"From": "admin@bank.com", "Reply-To":
"attacker@evil.com"},
        "body": "Urgent: Verify your banking password immediately
at http://192.168.0.5/auth"},
        {"headers": {"From": "dept@simats.edu", "Reply-To":
"dept@simats.edu"},
        "body": "Please find attached the timetable for the
upcoming assessment cycle."}
    ]

    preds = detector.predict(test_emails)
    classes = {1: "MALICIOUS (Phishing)", 0: "LEGITIMATE (Ham)"}
    for idx, p in enumerate(preds):
        print(f"Email {idx+1} Classification Result: {classes[p]}")

```


3. Model Effectiveness & Evaluation Analysis:

1. Hybrid Feature Synergy: Relying solely on keyword TF-IDF exposes models to adversarial synonym evasion. Augmenting NLP with structural heuristics (IP URLs, domain entropy, header mismatches) drastically reduces False Negatives.

2. Precision vs. Recall Trade-off: In email security, False Positives (marking critical legitimate business emails as phishing) cause severe disruption. Ensemble classifiers like Random Forest achieve high F1-score with balanced precision/recall.

3. Robustness Against Evasion: Attackers frequently use URL redirectors and zero-width spaces. Combining header metadata authentication (SPF/DKIM/DMARC status) alongside dynamic feature extraction provides defense-in-depth protection.